

# Taller 3 – Documentación programación discreta

Jair Gómez Narváez, Aimer Esteban García Rojas

Programa de Ingeniería de Sistemas

Universidad Nacional de Colombia

Julio 2026

## I. PROPÓSITO Y ALCANCE DEL DOCUMENTO

Este documento acompaña el repositorio de código correspondiente al Taller 3 de Matemáticas Discretas, para cada uno de los diez puntos propuestos en el enunciado se describe el problema que resuelve el programa correspondiente, la idea matemática en la que se apoya, la forma de ejecutarlo, las pruebas realizadas y las limitaciones identificadas.

El repositorio se organiza en cuatro carpetas dentro de src/cripto/ para los puntos 1 a 3, grafos/ para los puntos 4 a 6, boole/ para los puntos 7 a 9 y cuántica/ para el punto 10. Cada módulo puede ejecutarse de forma independiente y las pruebas automatizadas correspondientes se encuentran en la carpeta tests/.

## II. BLOQUE A. CRIPTOGRAFÍA

### II-A. Cifrado César: romper un mensaje antiguo

**Problema.** El programa cifra y descifra un texto desplazando cada letra una cantidad fija  $k$  de posiciones dentro del alfabeto y permite además recuperar un mensaje cuando no se conoce  $k$ , probando los 26 desplazamientos posibles.

**Idea matemática.** El cifrado se basa en aritmética modular sobre el alfabeto de 26 letras: representando cada letra por su posición  $A=0, \dots, Z=25$ , cifrar corresponde a la operación  $(\text{posición} + k) \bmod 26$ , y descifrar equivale a aplicar el desplazamiento inverso  $-k$  sobre el mismo módulo. Al existir solo 26 claves posibles, el cifrado es trivialmente vulnerable a un ataque de fuerza bruta que las prueba todas.

**Ejecución.** El módulo src/cripto/cesar.py se ejecuta con python y despliega un menú de consola con tres opciones: cifrar, descifrar y fuerza bruta. Los caracteres que no son letras se conservan sin modificar y se respeta la diferencia entre mayúsculas y minúsculas.

**Pruebas.** Se verificó el caso del enunciado cifrar "HOLA UNAL" con  $k = 3$  produce "KROD XQDO", la propiedad de que cifrar y luego descifrar con el mismo  $k$  reproduce el texto original y que el desplazamiento correcto aparece siempre entre las 26 opciones listadas por la fuerza bruta. Estas comprobaciones están automatizadas en tests/test\_cesar.py.

**Limitaciones.** La implementación cubre únicamente el alfabeto A-Z, sin la letra Ñ y no ofrece ninguna seguridad real dado el reducido espacio de claves.

**Nota:** El cifrado es una suma modular  $C = P + k \bmod 26$ , deshacerla requiere la operación inversa dentro de ese grupo aditivo, siendo esta la resta, es decir, equivalente a sumar  $-k$ . Por eso descifrar es técnicamente cifrar con un valor de  $-k$ . La fuerza bruta es posible puesto que el espacio de claves es muy pequeño, solo hay 26 valores posibles para  $k$ , así que en un caso determinado, un atacante podría probar todas las posibilidades y reconocer a simple vista cuál produce un texto con sentido.

### II-B. RSA de juguete: llaves, cifrado y descifrado

**Problema.** El programa implementa una versión sencilla del algoritmo RSA, permitiendo generar las claves a partir de dos números primos, cifrar un mensaje entero y recuperarlo mediante el proceso de descifrado.

**Idea matemática.** A partir de dos números primos  $p$  y  $q$  se calcula  $n=pq$  y  $\phi(n)=(p-1)(q-1)$ . Luego se obtiene el inverso modular  $d$  del exponente público  $e$  mediante el algoritmo de Euclides Extendido. El cifrado y descifrado se realizan utilizando exponenciación modular.

**Ejecución.** El módulo src/cripto/rsa.py solicita al usuario los valores de  $p$ ,  $q$ ,  $e$  y el mensaje. El programa valida que los números sean correctos, calcula las claves, cifra el mensaje y finalmente lo descifra mostrando todos los parámetros obtenidos.

**Pruebas.** Se verificó el caso obligatorio del taller con  $p=61$ ,  $q=53$ ,  $e=17$  y  $M=65$ , obteniendo los valores esperados. También se probaron casos con números no primos, exponentes inválidos y mensajes fuera del rango permitido mediante tests/test\_rsa.py

**Limitaciones.** La implementación utiliza números pequeños con fines educativos y no ofrece seguridad real. Además, solo permite cifrar mensajes enteros menores que  $nnn$ , sin aplicar técnicas de relleno o manejo de textos completos.

**Nota:** Los números primos  $p$  y  $q$  permiten construir los valores  $n$  y  $\phi(n)$ , que son la base para generar la llave pública y privada del algoritmo RSA. El inverso modular se utiliza para calcular el exponente privado  $d$ , el cual permite

recuperar el mensaje original después de haber sido cifrado, mientras que la congruencia modular es el principio matemático sobre el que funciona RSA, ya que tanto el cifrado como el descifrado se realizan mediante potencias módulo  $n$ , asegurándonos que el mensaje cifrado pueda convertirse nuevamente en el mensaje original al aplicar la clave correspondiente.

---

### II-C. MPC básico: calcular un promedio sin mostrar los datos

**Problema.** El programa simula un protocolo de computación multipartita segura con tres servidores, que permite calcular la suma y el promedio de una lista de notas sin que ningún servidor individual conozca los valores originales.

**Idea matemática.** Cada nota  $x$  se reparte en tres números aleatorios  $a$ ,  $b$ ,  $c$  tales que  $(a + b + c) \bmod M = x$ , con  $M = 1000003$ . Cada servidor recibe solo una de las tres partes de cada nota, de modo que ninguno puede reconstruir el valor original por separado; al sumar las tres partes parciales módulo  $M$  se recupera la suma total, de la que se obtiene el promedio.

**Ejecución.** El módulo `src/cripto/mpc.py`, ofrece la opción de revisar el ejemplo del enunciado o de ingresar una lista propia de notas enteras entre 0 y 50, mostrando la vista parcial que vería cada servidor y el resultado reconstruido.

**Pruebas.** El caso del taller, notas siendo [40, 35, 50, 25] produce una suma de 150 y un promedio de 37.5, verificado con una aserción dentro del propio programa y en `tests/test_mpc.py`. También se probó el manejo de notas fuera de rango y de listas vacías, en ambos casos con un error de forma controlada.

**Limitaciones.** El módulo  $M$  se fijó en 1000003, con listas más grandes o valores distintos habría que ajustarlo para evitar colisiones. Además, el reparto aleatorio se simula dentro de un mismo proceso y no involucra una comunicación real entre servidores independientes.

**Nota:** Por ejemplo con  $x = 10$  y  $M = 1\,000\,003$ , pues se generan al azar  $s1 = 784512$  y  $s2 = 215023$ , se calcula  $s3 = x - s1 - s2 \bmod M = 478$ . Cada parte por separado siendo 784512, 215023 o 478, es indistinguible de un número aleatorio cualquiera entre 0 y  $M-1$ , por lo que no revela nada sobre  $x$ . Únicamente al sumar las tres módulo  $M$  se recupera el valor original:  $784512 + 215023 + 478 \bmod 1\,000\,003 = 10$ .

---

## III. BLOQUE B. GRAFOS

### III-A. Ruta más corta: una ciudad como grafo

**Problema.** Encuentra la ruta de menor costo entre dos puntos de una red de transporte modelada como grafo ponderado, devolviendo tanto la distancia total como la secuencia de vértices que conforman la ruta.

**Idea matemática.** Se implementó el algoritmo de Dijkstra con una cola de prioridad, partiendo del origen con distancia 0, se expande en cada paso el vértice con menor distancia tentativa conocida. Como el grafo no tiene pesos negativos, una vez marcado un vértice como visitado su distancia ya es mínima, lo que garantiza un resultado acertado.

**Ejecución.** El módulo `src/grafos/dijkstra.py` carga un grafo de ejemplo de 8 vértices y 12 aristas desde un archivo CSV en `src/grafos/datos/` y permite ver el ejemplo predefinido, ingresar un origen y destino propios o abrir la interfaz gráfica `dijkstra_visual.py` para construir el grafo de forma interactiva.

**Pruebas.** Se verificó el caso de ejemplo Portal → Estadio, con distancia esperada 12 y ruta Portal→Museo→Calle26→Centro→Estadio, además de la consulta con vértices inexistentes, que produce un error controlado. Las pruebas están automatizadas en `tests/test_dijkstra.py`.

**Limitaciones.** El algoritmo asume pesos no negativos y no funcionaría correctamente si se introdujeran aristas de peso negativo. El grafo de prueba es pequeño y pensado para visualización, no se evaluó el rendimiento en redes de gran escala.

**Nota:** El algoritmo va marcando como si fuese la ruta definitiva la menor distancia acumulada conocida en cada paso, asumiendo que ya no puede mejorar después, sin embargo, esto solo es posible si todos los pesos son mayores o iguales a 0, pues con un peso negativo, una ruta ya cerrada podría "abaratarse" más adelante y perdería todo el sentido. Un camino óptimo es aquel cuya suma total de los pesos de las aristas recorridas es la mínima posible entre todas las rutas que conectan un origen y un destino.

---

### III-B. Cierre de una estación: medir el impacto en la red

**Problema.** Mide el impacto de cerrar un vértice o una arista de la red, por lo que recalcula las rutas más cortas entre varios pares origen destino antes y después del cierre y reporta cuáles aumentaron su distancia y cuáles quedaron desconectados.

**Idea matemática.** Se reutiliza el algoritmo de Dijkstra del punto anterior sobre dos versiones del grafo, la original y una copia a la que se elimina un vértice junto con todas sus aristas. Comparar las distancias par a par antes y después del

cierre permite cuantificar el efecto estructural sobre la conexión de la red.

**Ejecución.** El módulo `src/grafos/cierre.py` expone las funciones `eliminar_vertice()` y `eliminar_arista()`, que devuelve un grafo nuevo sin modificar el original, y una función `comparar_rutas()` que arma la tabla de resultados con origen, destino, distancia antes, distancia después, diferencia y estado.

**Pruebas.** Se probó el cierre de un vértice central, verificando que las rutas que lo atravesaban aumentaran su distancia o quedaran marcadas como desconectadas. Las pruebas están automatizadas en `tests/test_cierre.py`.

**Limitaciones.** El análisis se realiza par a par mediante ejecuciones independientes de Dijkstra, lo cual no escala bien si se quisiera evaluar el cierre de muchos vértices distintos sobre grafos de gran tamaño.

**Nota:** En la prueba se cerró el vértice Centro, ya que es uno de los nodos con mayor cantidad de conexiones dentro del grafo. Este vértice comunica directamente con Calle26, Museo, Universidad, Parque y Estadio, por lo que hace parte de varios de los caminos más cortos entre diferentes estaciones, como resultado, las cinco rutas evaluadas aumentaron su distancia, con incrementos entre 2 y 9 unidades, aunque ninguna quedó completamente desconectada, los recorridos se volvieron menos eficientes, lo que demuestra que Centro es un punto clave dentro de la red y que su eliminación afectaría el costo de múltiples trayectos, esto aplicado a la vida real podría verse como el cierre de una avenida principal.

---

### III-C. Coloreo de grafos: organizar exámenes sin choques

**Problema.** Asigna un color, representando una franja horaria a cada curso de modo que dos cursos con estudiantes en común, buscando que los vértices adyacentes nunca queden en la misma franja y reporta cuántos colores se usaron y qué vértices quedaron en cada uno.

**Idea matemática.** Se implementó un algoritmo voraz de coloreo, en la variante de Welsh Powell, se recorren los vértices y a cada uno se le asigna el primer color no usado por ninguno de sus vecinos ya coloreados. Esta estrategia siempre produce una asignación válida, aunque no garantiza el número mínimo de colores en el grafo.

**Ejecución.** El módulo `src/grafos/coloreo.py` carga un grafo de conflictos de al menos 10 vértices desde un archivo CSV, ejecuta el algoritmo voraz y verifica que ningún par de vértices adyacentes comparta color antes de mostrar el resultado agrupado por color.

**Pruebas.** Se verificó de forma automática que ningún par de vértices adyacentes recibiera el mismo color, sobre el grafo de prueba de mínimo 10 vértices interpretado como cursos con estudiantes cruzados. Las pruebas están en `tests/test_coloreo.py`.

**Limitaciones.** Al ser un algoritmo voraz, el número de colores usados depende del orden en que se procesan los vértices y no siempre corresponde al número mínimo teórico del grafo.

**Nota:** El algoritmo voraz asigna colores siguiendo un orden fijo de vértices, dando a cada uno el primer color no usado por sus vecinos ya coloreados, el problema se genera puesto que nunca reconsidera esa decisión. Por eso el orden de procesamiento puede forzar el uso de colores adicionales que un análisis global podría haber evitado, por lo que no se garantiza un número mínimo de colores, sin embargo, sí garantiza validez porque en cada paso, solo asigna un color que ningún vecino ya coloreado esté usando, así que nunca puede quedar un par de vértices adyacentes compartiendo color, sin importar el orden.

---

## IV. BLOQUE C. ÁLGEBRA DE BOOLE, SHANNON Y COMPUTACIÓN CUÁNTICA

### IV-A. Tablas de verdad y circuitos lógicos

**Problema.** Genera la tabla de verdad completa de una expresión booleana con variables A, B y C, mostrando el resultado para cada una de las combinaciones posibles de entrada.

**Idea matemática.** Una expresión booleana de n variables define una función. Recorriendo todas las combinaciones posibles de entrada mediante producto cartesiano y evaluando la expresión en cada una, se obtiene la tabla completa, equivalente a la tabla de verdad de un circuito digital.

**Ejecución.** El módulo `src/boole/tablas.py` define como funciones de Python las expresiones exigidas en el enunciado, estas siendo  $(A \wedge B) \vee (\neg C)$ ,  $(A \oplus B) \wedge C$ ,  $(A \vee B) \wedge (\neg A \vee C)$ , se usa `itertools.product` para generar las 8 combinaciones de A, B y C e imprimir la tabla de cada una.

**Pruebas.** Se generaron las tablas completas de las tres expresiones obligatorias y se verificaron manualmente algunas filas contra la evaluación booleana esperada. Las pruebas automatizadas están en `tests/test_tablas.py`.

**Limitaciones.** La evaluación está implementada para expresiones fijas definidas como funciones de Python y no para un analizador de expresiones booleanas escritas como texto libre por el usuario.

**Nota:** Cada variable de la expresión corresponde a una señal de entrada del circuito, apagado o encendido = 1 o 0, cada operador lógico AND, OR, NOT, XOR corresponde a una compuerta que combina esas señales. La tabla de verdad enumera lo que son todas las combinaciones posibles de entrada junto con la salida que produciría el circuito en cada una, por lo que construir la tabla de verdad de una expresión equivale a simular exhaustivamente el comportamiento del circuito que esa expresión representa.

---

#### *IV-B. Simplificación booleana: hacer un circuito más barato*

**Problema.** El programa recibe una función booleana representada mediante minterminos y obtiene una expresión equivalente simplificada en forma de suma de productos, verificando posteriormente que ambas expresiones produzcan la misma tabla de verdad.

**Idea matemática.** Se implementó una versión simplificada del algoritmo de Quine-McCluskey. Los minterminos se agrupan según la cantidad de unos en su representación binaria, se combinan aquellos que difieren en un solo bit y posteriormente se identifican los implicantes primos y esenciales para construir una expresión equivalente.

**Ejecución.** El módulo `src/boole/simplificacion.py` permite ingresar el número de variables y los minterminos de la función. El programa muestra los grupos generados, los implicantes obtenidos, la tabla de cobertura y la expresión simplificada.

**Pruebas.** Se probaron funciones de tres y cuatro variables, incluyendo el caso sugerido por el taller con los minterminos  $\{1,3,5,7\}$ . Además, `tests/test_simplificacion.py` verifica funciones constantes, un solo mintermino y expresiones de mayor tamaño.

**Limitaciones.** La implementación utiliza una selección voraz de implicantes y no implementa el método completo de Petrick, por lo que en algunos casos puede obtener una expresión equivalente, aunque no necesariamente la mínima posible.

**Nota:** Un mintermino es una operación AND que incluye todas las variables de una función, tanto directas como negadas, que vale 1 para exactamente una combinación de entrada. Numerándolos según el valor binario de esa combinación, cualquier función booleana puede describirse como la suma de sus minterminos, es decir, indicando en qué filas exactas de la tabla vale 1. Dos expresiones son equivalentes si producen el mismo valor de salida para cada combinación de entradas posibles generan la misma tabla de verdad, sin importar cuántos operadores o qué forma algebraica use cada una para llegar a ese resultado.

---

#### *IV-C. Shannon: medir información en un mensaje*

**Problema.** El programa calcula la entropía de Shannon de un mensaje, obteniendo la frecuencia y probabilidad de cada símbolo, además de comparar dos mensajes para determinar cuál contiene mayor cantidad de información.

**Idea matemática.** Primero se cuentan las apariciones de cada símbolo y se calcula su probabilidad de ocurrencia. Posteriormente se aplica la fórmula de la entropía de Shannon para medir el nivel de incertidumbre del mensaje. Finalmente se comparan dos textos para identificar cuál presenta una distribución de símbolos más variada.

**Ejecución.** El módulo `src/informacion/entropia.py` solicita un mensaje, calcula sus frecuencias, probabilidades y entropía. Después compara automáticamente dos mensajes y explica cuál posee mayor entropía y la razón de este resultado.

**Pruebas.** Se verificaron mensajes con un único símbolo, símbolos equiprobables, distribuciones desiguales y mensajes repetitivos frente a mensajes variados mediante `tests/test_entropia.py`.

**Limitaciones.** La implementación trabaja únicamente con la frecuencia de los símbolos individuales y no considera dependencias entre caracteres consecutivos ni modelos de compresión más avanzados, como Huffman.

**Nota:** La entropía mide la incertidumbre y no la longitud del texto, porque el tamaño de un mensaje no refleja cuánta información real contiene, sino qué tan difícil es adivinarlo. Un mensaje larguísimo que repite la misma letra un millón de veces no aporta ninguna sorpresa porque ya sabemos qué carácter sigue, por lo que su incertidumbre y su entropía son casi nulas, en cambio, una contraseña de por ejemplo, ocho caracteres, es muy corta, pero es imposible de anticipar porque cada símbolo es completamente diferente, lo que genera una entropía máxima.

---

#### *IV-D. Primer simulador cuántico: bits, qubits y mediciones*

**Problema.** El programa simula el comportamiento básico de un qubit, permitiendo aplicar las compuertas cuánticas X, Z y H, calcular las probabilidades de medición y realizar una simulación de múltiples mediciones.

**Idea matemática.** El estado del qubit se representa mediante un vector de dos componentes. Cada compuerta cuántica corresponde a una matriz de  $2 \times 2$  que transforma dicho vector mediante multiplicación matricial. Las probabilidades de medir los estados  $|0\rangle$  y  $|1\rangle$  se obtienen elevando al cuadrado el valor absoluto de cada amplitud.

**Ejecución.** El módulo `src/cuantica/qubit.py` inicia el qubit en el estado  $|0\rangle$ , permite ingresar una secuencia de compuertas, muestra el estado resultante, calcula las probabilidades de medición y simula 1000 mediciones reportando las frecuencias observadas.

**Pruebas.** Se verificaron los casos obligatorios del taller:  $X|0\rangle=|1\rangle$ ,  $H|0\rangle$  produce probabilidades cercanas al 50 % para ambos estados y  $HH|0\rangle=|0\rangle$ . Además, se comprobó el funcionamiento de la compuerta  $Z$  mediante `tests/test_qubit.py`.

**Limitaciones.** La simulación representa únicamente un qubit aislado y utiliza números reales. No considera amplitudes complejas, sistemas de varios qubits, entrelazamiento ni la ejecución sobre un computador cuántico real.

**Nota:** La diferencia se encuentra en que la simulación clásica es un cálculo matemático exacto donde la computadora calcula las probabilidades de forma mediante álgebra lineal, es solo generar números aleatorios artificiales basados en porcentajes ya conocidos, mientras que en un computador cuántico real el estado no se puede calcular ni leer directamente, sino que cada medición colapsa físicamente al qubit a 0 o 1 de forma probabilística, obligando a repetir el experimento miles de veces sobre un hardware real que a diferencia de un cálculo en software ideal, está sujeto a ruido, errores e incoherencias.

---